



실습 워크북 : 스톱워치



학습 목표

- `useState` 로 시간/동작 상태 관리하기
- `useEffect` 로 타이머 시작·정지·정리(`cleanup`) 구현하기
- `useMemo` 로 표시 문자열(`mm:ss`) 포맷 계산 최적화
- `useCallback` 으로 핸들러 함수 안정화
- **Tailwind/DaisyUI**로 기본 UI 스타일링 (수업 범위 내 유틸리티만)



완성 스펙(디자인 가이드)

- 상단 제목: 굵고 크게
- 중앙에 **mm:ss** 형식 시간 표시(큰 글씨)
- 버튼 3개: 시작 / 일시정지 / 리셋
- 기본 여백, 테두리, 라운드, 버튼 간 간격은 **margin**으로 처리 (`mr-2` , `mb-2` , `p-2` , `border` , `rounded`)
- 레이아웃: 가운데 정렬
 - 바깥 컨테이너: `w-full flex flex-col items-center p-6`
 - 내부 카드: `w-80 p-4 border rounded`

0) 프로젝트 준비

```
npm create vite@latest stopwatch -- --template react-ts
cd stopwatch
npm i
```

```
npm i -D tailwindcss @tailwindcss/postcss postcss daisyui
```

`App.tsx` 대신 `Stopwatch.tsx` 를 만들어서 실습합니다.

1) UI 뼈대 + 스타일 (먼저 화면부터)

아래 코드를 그대로 붙여넣고, TODO를 채워가며 기능을 완성합니다.

```
import { useState, useEffect, useMemo, useCallback } from "react";

export default function Stopwatch() {
  // ● 상태
  const [seconds, setSeconds] = useState(0);
  const [running, setRunning] = useState(false);

  // ● 핸들러 (아래에서 구현)
  const start = useCallback(() => {
    // TODO: running을 true로
  }, []);

  const pause = useCallback(() => {
    // TODO: running을 false로
  }, []);

  const reset = useCallback(() => {
    // TODO: running false + seconds 0으로
  }, []);

  // ● 타이머 동작 (아래 useEffect에서 구현)

  // ● 표시 문자열 (mm:ss)
  const display = useMemo(() => {
    // TODO:// 1) 분 = Math.floor(seconds / 60)
    // 2) 초 = seconds % 60
    // 3) 두 자리로 맞추기: String(n).padStart(2, "0")
    // 4) "mm:ss"로 반환
    return "00:00";
  }, [seconds]);

  return (
    <div className="w-full flex flex-col items-center p-6">
      <div className="w-80 p-4 border rounded">
        <h1 className="text-xl font-bold mb-2">🕒 Stopwatch</h1>
      </div>
    </div>
  );
}
```

```

    {/* 시간 표시 */}
    <div className="mb-2" style={{ fontSize: 40, textAlign: "center" }}>
      {display}
    </div>

    {/* 버튼들 */}
    <div className="mb-2" style={{ textAlign: "center" }}>
      {/* gap 대신 margin-right로 간격 */}
      <button className="btn btn-primary mr-2" onClick={start}>시작</button>
      <button className="btn btn-outline mr-2" onClick={pause}>일시정지</button>
      <button className="btn" onClick={reset}>리셋</button>
    </div>

    {/* 상태 표시(선택) */}
    <p className="text-sm" style={{ textAlign: "center" }}>
      상태: {running ? "동작 중" : "정지"}
    </p>
  </div>
</div>
);
}

```

💡 DaisyUI를 사용하지 않는다면 버튼 클래스들(btn, btn-primary, btn-outline) 대신 `className="p-2 border rounded mr-2"` 처럼 Tailwind 기본 유틸리티만 써도 됩니다.

2) 타이머 동작 구현 (useEffect)

- `running === true` 일 때만 `setInterval` 로 1초마다 `setSeconds(s ⇒ s + 1)` 호출
- 정리(cleanup)에서 `clearInterval` 로 해제 (메모리 누수 방지)
- `running` 값이 바뀔 때마다 타이머를 재관리

```

useEffect(() ⇒ {
  if (!running) return;

```

```
const id = setInterval(() => {
  setSeconds(s => s + 1);
}, 1000);

return () => clearInterval(id);
}, [running]);
```

3) 표시 문자열 계산 (useMemo)

- 분/초 계산 후 두 자리로 맞춰서 "mm:ss" 문자열 생성
- seconds가 바뀔 때만 다시 계산

```
const display = useMemo(() => {
  const mm = Math.floor(seconds / 60);
  const ss = seconds % 60;
  const mmStr = String(mm).padStart(2, "0");
  const ssStr = String(ss).padStart(2, "0");
  return `${mmStr}:${ssStr}`;
}, [seconds]);
```

4) 핸들러 구현 (useCallback)

```
const start = useCallback(() => {
  setRunning(true);
}, []);

const pause = useCallback(() => {
  setRunning(false);
}, []);

const reset = useCallback(() => {
  setRunning(false);
  setSeconds(0);
}, []);
```

5) 체크리스트

- ☐ UI가 가운데 정렬되고, 카드(`w-80 p-4 border rounded`) 안에 표시되는가?
 - ☐ 시작 클릭 시 초가 증가하고, 일시정지 시 멈추는가?
 - ☐ 리셋 클릭 시 00:00으로 돌아가고 멈추는가?
 - ☐ `cleanup`이 작동해 탭 이동 등에서도 타이머가 남지 않는가?
 - ☐ `gap-*` 없이 `mr-2` / `mb-2` 등으로 간격을 주었는가?
-

6) 확장 과제

1. 랩타임 기록: "랩" 버튼을 눌러 현재 시간을 리스트에 쌓기
 - `const [laps, setLaps] = useState<number[]>([])`
 - `setLaps(prev => [...prev, seconds])`
 - 리스트 렌더링 + 포매팅은 `display` 계산 로직 재사용
 2. 시간 포맷 확장: 10분 이상이면 `hh:mm:ss` 로 자동 전환
 3. 버튼 상태: 동작 중일 때는 "시작" 버튼 비활성화, 정지일 때는 "일시정지" 비활성화
 - `disabled={running}` / `disabled={!running}`
 4. 키보드 단축키: Space = 시작/정지 토글, R = 리셋 (추가 `useEffect` 로 keydown 이벤트)
-

실행결과

